

Unit -9

Introduction to Crash Recovery and Its Importance

What is Crash Recovery?

Crash recovery helps restore a computer system or database to a correct and stable state after a failure. It ensures that data is not lost or corrupted due to crashes. This is essential for maintaining system reliability and preventing data losses.

Crash recovery is a process that helps a system recover after a failure. It includes:

1. **Detecting Failure** – Identifying crashes caused by hardware issues, software bugs, or power failures.
2. **Protects Data** – It makes sure no important data is lost or damaged.
3. **Restoring System** – Bringing the system back to a stable state by undoing incomplete actions and applying updates.

Importance of Crash Recovery

Crash recovery is vital for several reasons:

1. **Data Integrity:** Ensures that data remains accurate and reliable even after a failure. This is crucial for applications where data accuracy is paramount, such as banking, healthcare, and e-commerce.
2. **System Reliability:** Enhances the overall reliability of the system by allowing it to recover from failures without significant data loss or corruption. Users can trust that their data is safe and the system will remain operational.
3. **Minimal Downtime:** It helps the system get back to work quickly after a crash. This keeps services running with little delay, so users are not affected for long.
4. **User Trust:** Users feel confident in the system because it can recover from crashes without losing data or stopping services.

Failure Classification

To find that where the problem has occurred, we generalize a failure into the following categories:

1. Transaction failure
2. System crash
3. Disk failure

1. Transaction failure

The transaction failure occurs when it fails to execute or when it reaches a point from where it can't go any further. If a few transaction or process is hurt, then this is called as transaction failure.

Reasons for a transaction failure could be -

1. Logical errors: If a transaction cannot complete due to some code error or an internal error condition, then the logical error occurs.

2. Syntax error: This happens when the database system stops a running transaction because it cannot complete it. For example, the system may stop a transaction if there is a deadlock or a lack of resources.

2. System Crash

- System failure can occur due to power failure or other hardware or software failure. Example: Operating system error.

3. Disk Failure

Common problem in the early days of technology evolution.

- Disk failure occurs due to the formation of bad sectors, disk head crash, and unreachability.
- It occurs in situations where hard-disk drives or storage devices fail frequently. Such failures may be caused by disk or other hardware issues, which can destroy all or part of the stored data.

Storage structure:

1. Volatile storage: Volatile storage is a type of memory that loses all data if the system crashes or turns off. It is placed very close to the CPU, often built into the chipset.

Examples:

Main Memory (RAM) – Helps run programs but clears when power is lost.

Cache Memory – Stores frequently used data for quick access.

It works very fast but can only store a small amount of data.

2. Non-volatile storage: This type of memory keeps data even when the system is turned off or crashes. It can store a large amount of information but is slower to access. Examples include hard drives, magnetic tapes, and flash memory.

Recovery and Atomicity:

When a system crashes, many transactions may be running and files may be open. Each transaction has multiple steps that must all be completed together (atomicity). According to the ACID rules, either all steps of a transaction happen, or none do.

During recovery, the database system must:

- Check the status of all ongoing transactions.
- Ensure that any transaction in progress maintains atomicity.
- Decide whether a transaction was completed or needs to be rolled back.
- Prevent any transaction from leaving the database in an inconsistent state.

There are 2 forms of techniques, which may facilitate a database management System in recovering as well as maintaining the atomicity of a transaction:

1. **Log-based Recovery** – Uses logs to track changes and restore data if needed.
2. **Shadow Paging** – Maintains copies of data to ensure a safe rollback.

Log-Based Recovery:

- The log is a sequence of records. Log of each transaction is maintained in some stable storage so that if any failure occurs, then it can be recovered from there.
- If any operation is performed on the database, then it will be recorded in the log.
- But the process of storing the logs should be done before the actual transaction is applied in the database.

Let's assume there is a transaction to modify the City of a student. The following logs are written for this transaction.

- When the transaction is initiated, then it writes 'start' log.

1. <In, Start>

- When the transaction modifies the City from 'Noida' to 'Bangalore', then another log is written to the file.

1. <In, City, 'Noida', 'Bangalore' >

When the transaction is finished, then it writes another log to indicate the end of the transaction.

1. <In, Commit>

There are two approaches to modify the database:

1. Deferred database modification:

- Database is not modified until the transaction commits.
- During execution, only log records are written to stable storage.
- Actual updates are applied after commit.
- UNDO is not required during recovery.
- Only REDO is needed if a failure occurs.

$\langle T_0 \text{ start} \rangle$	$\langle T_0, \text{start} \rangle$	$\langle T_0 \text{ start} \rangle$
$\langle T_0, A, 950 \rangle$	$\langle T_0, A, 950 \rangle$	$\langle T_0, A, 950 \rangle$
$\langle T_0, B, 2050 \rangle$	$\langle T_0, B, 2050 \rangle$	$\langle T_0, B, 2050 \rangle$
	$\langle T_0 \text{ commit} \rangle$	$\langle T_0 \text{ commit} \rangle$
	$\langle T_1 \text{ start} \rangle$	$\langle T_1 \text{ start} \rangle$
	$\langle T_1, C, 600 \rangle$	$\langle T_1, C, 600 \rangle$
		$\langle T_1 \text{ commit} \rangle$
(a)	(b)	(c)

↳ If log on stable storage at time of crash is given above (i.e. a, b, c)

- * No redo actions need to be taken for case 'a'
- * redo(T_0) must be performed for case b since $\langle T_0 \text{ commit} \rangle$ is present.
- * redo(T_0) must be performed followed by redo(T_1) since $\langle T_0 \text{ commit} \rangle$ and $\langle T_1 \text{ commit} \rangle$ are present.

2. Immediate database modification:

- In the Immediate modification technique, database updates are allowed before the transaction commits, i.e., while the transaction is still active.
- However, changes are first written to the log, and then they may be written to the database (Write-Ahead Logging is followed).
- The actual database may be modified immediately after each operation, but this is not mandatory, it depends on the buffer management and scheduling.
- Since updates can occur before commit, both UNDO and REDO are required during recovery:
- **UNDO:** if the transaction aborts
- **REDO:** if the transaction commits but changes are not yet reflected on disk

Example:

a.

$\langle T_0 \text{ start} \rangle$

Undo(T_0):

$\langle T_0, A, 1000, 950 \rangle$ B is restored to 2000

$\langle T_0, B, 2000, 2050 \rangle$ and A to 1000

b. $\langle T_0 \text{ start} \rangle$

$\langle T_0, A, 1000, 950 \rangle$

Undo(T_1) and redo(T_0):

$\langle T_0, B, 2000, 2050 \rangle$

C is restored to 700,

$\langle T_0 \text{ commit} \rangle$

and then A and B are

$\langle T_1 \text{ start} \rangle$

set to 950 and 2050

$\langle T_1, C, 700, 600 \rangle$

respectively

c. $\langle T_0 \text{ start} \rangle$

$\langle T_0, A, 1000, 950 \rangle$

Redo(T_0) and redo(T_1):

$\langle T_0, B, 2000, 2050 \rangle$

A and B are set to

$\langle T_0 \text{ commit} \rangle$

950 and 2050 respectively.

$\langle T_1 \text{ start} \rangle$

Then C is set

$\langle T_1, C, 700, 600 \rangle$

to 600.

$\langle T_1 \text{ commit} \rangle$

Recovery using Log records

When the system is crashed, then the system consults the log to find which transactions need to be undone and which need to be redone.

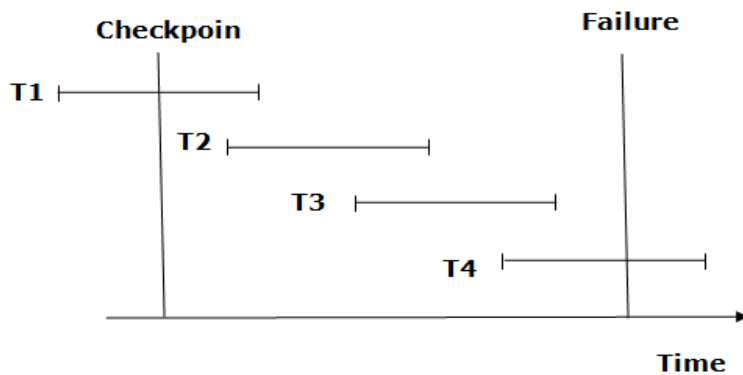
1. If the log contains the record $\langle T_i, \text{Start} \rangle$ and $\langle T_i, \text{Commit} \rangle$ then the Transaction T_i needs to be redone.
2. If log contains record $\langle T, \text{Start} \rangle$ but does not contain the record either $\langle T_i, \text{Commit} \rangle$ or $\langle T_i, \text{abort} \rangle$, then the Transaction T_i needs to be undone.

Checkpoint

- The checkpoint is a type of mechanism where all the previous logs are removed from the system and permanently stored in the storage disk.
- The checkpoint is like a bookmark. While the execution of the transaction, such checkpoints are marked, and the transaction is executed then using the steps of the transaction, the log files will be created.
- When the system reaches a checkpoint, all completed transactions are saved in the database. After that, the old log file is deleted, and a new log file starts recording transactions until the next checkpoint. This process keeps repeating.
- The checkpoint is used to declare a point before which the DBMS was in the consistent state, and all transactions were committed.

Recovery using Checkpoints

In the following manner, a recovery system recovers the database from this failure:

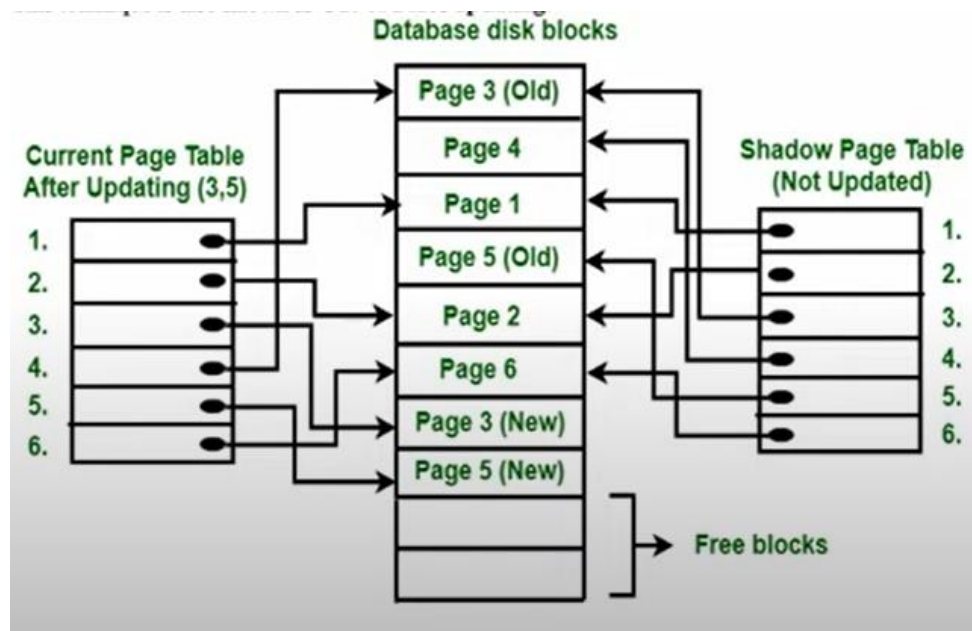


- The recovery system reads log files from the end to start. It reads log files from T4 to T1.
- Recovery system maintains two lists, a redo-list, and an undo-list.
- The transaction is put into redo state if the recovery system sees a log with $\langle T_n, \text{Start} \rangle$ and $\langle T_n, \text{Commit} \rangle$ or just $\langle T_n, \text{Commit} \rangle$. All transactions in the redo list are redone to ensure their committed changes appear in the database.
- **For example:** In the log file, transaction T2 and T3 will have $\langle T_n, \text{Start} \rangle$ and $\langle T_n, \text{Commit} \rangle$. The T1 transaction will have only $\langle T_n, \text{commit} \rangle$ in the log file. That's why the transaction is committed after the checkpoint is crossed. Hence it puts T1, T2 and T3 transaction into redo list.
- The transaction is put into undo state if the recovery system sees a log with $\langle T_n, \text{Start} \rangle$ but no commit or abort log found. In the undo-list, all the transactions are undone, and their logs are removed.
- Transaction T4 will have $\langle T_n, \text{Start} \rangle$. So T4 will be put into undo list since this transaction is not yet complete and failed amid.

Shadow Paging:

Shadow Paging is a database recovery technique that ensures data consistency without using logs. Here's how it works:

1. **Database Storage:** The database is divided into fixed-size units called pages, which are stored on disk.
2. **Page Tables:** There are two tables that keep track of these pages:
 - **Current Page Table:** Points to the latest version of the database.
 - **Shadow Page Table:** A backup copy of the page table before any changes.
3. **Transaction Process:**
 - When a transaction begins, a copy of the current page table is saved as the shadow page table.
 - Changes are made only to the current page table, while the shadow page table remains unchanged.
 - If the transaction is successful, the system replaces the old shadow page table with the updated page table.
 - If the transaction fails, the system discards the changes and uses the old shadow page table, ensuring data remains safe.



To understand the concept, consider above figure. In this 2 write operations are performed on page 3 and 5. Before start of write operation on page 3, current page table points to old page 3. When write operation starts following steps are performed:

1. Firstly, search start for available free block in disk blocks.

2. After finding free block, it copies page 3 to free block which is represented by Page 3 (New).
3. Now the current page table points to Page 3 (New) on disk but shadow page table points to old page 3 because it is not modified.
4. The changes are now propagated to Page 3 (New) which is pointed by the current page table.

COMMIT Operation:

When a transaction is committed, the changes become permanent. Here's what happens:

1. Save Changes to Database

- All the changes made during the transaction are saved from temporary memory (buffer) to the actual database on disk.

2. Save the Updated Page Table

- The system saves the updated **page table** (which tracks the new data locations) to the disk.

3. Update the Safe Location

- The system updates a fixed storage location to point to the new page table.
- This replaces the old shadow page table with the new one.

Failure:

If the system crashes before the transaction is committed:

- The system ignores the changes and goes back to the shadow page table (backup).
- The modified pages are deleted, and everything returns to the way it was before the transaction started.

Advantages.

- This method requires fewer disk accesses to perform operation.
- In this method, recovery from crash is inexpensive and quite fast.
- There is no need of operations like- Undo and Redo.

Disadvantages:

- **Scattered Data:** When data is updated, it moves to a new location, making it harder to keep related data together.
- **Wasted Space:** old pages remain until commit and take up space.
- **Slower Transactions:** multiple page copies may need to be written at commit, which can slow down execution.

Backup and Recovery:

Backup and Recovery both terms are data protection terminology. An efficient backup and recovery system is critical for any firm to secure its precious data. However, backup and recovery are distinct techniques, backup stores a copy of the complete database onto storage media. In contrast, recovery is the process of retrieving lost data from backup storage mediums.

Backup

Backup refers to the process of creating copies of data to safeguard it against data loss. There are several types of database backup:

1. Full Backup

It is a simple and full back up procedure that copies all your data to another media set, including a tape, disk, or CD. As a result, a complete copy of all your data is provided on a single media set. It takes a longer time to complete and takes much storage space.

2. Incremental Backup

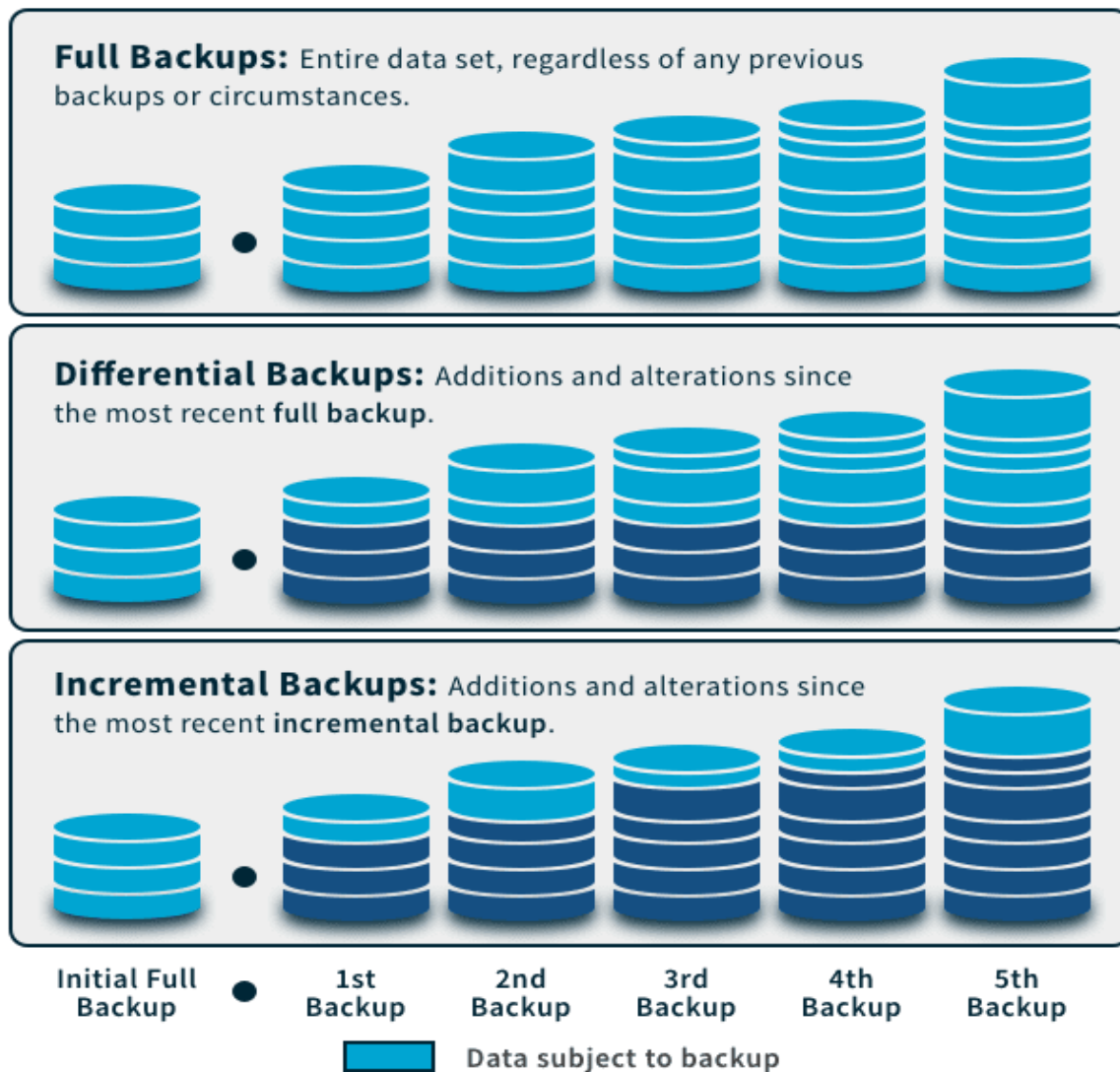
- Saves only the changes made since the last backup (full or incremental).
- Uses less space and time to back up.
- But takes the longest time to restore everything, because it needs the full backup + all incremental backups.
- Best for data that doesn't change often.

- Unpredictable size: You can't tell in advance how much space future backups will take.

3. Differential Backup

- Saves all the changes made after the last full backup.
- Grows larger over time until the next full backup.
- Faster to restore than incremental backups—only need the full backup the latest differential backup.
- Good for quick recovery without rebuilding everything.
- Useful for databases or servers that need fast restoration.

TYPES OF BACKUP: FULL, DIFFERENTIAL, AND INCREMENTAL



Recovery:

- Recovery refers to restoring the lost data in case of failure.
- So, we can say Recovery is a process of retrieving lost, corrupted or damaged data to its original state.
- In simple recovery is the process to store the database.
- The prior goal of recovery is retrieving original data in case of original data failure.
- It helps in improving the reliability of the database.
- Recovery has no role in data backup.

- An example of Recover can be Snap Manager recovers the data to the last transaction.
- Recovery aims to guarantee the atomicity of the transaction and data.

Assignments

Question: Explain the concept of crash recovery and discuss its importance in maintaining data integrity and system reliability.

Question: Describe the different types of system failures. How does classifying failures help in designing effective recovery strategies?

Question: Explain the backup and recovery process. What are the different types of backups, and how do they help in restoring data after a failure?

Question: Describe the storage hierarchy and explain how it affects data recovery and system performance.

Question: Explain the transaction model in database systems. How do transactions ensure consistency and reliability during system failures?

Question: Explain log-based recovery. How does maintaining a log help in recovering a database after a crash?

Question: Explain the shadow paging technique for crash recovery. Compare it with log-based recovery in terms of advantages and disadvantages.